

Tiny Second-hand Shopping Platform



화이트햇스쿨 4 기

23 반 오지송 (2562)

[GitHub Repository: Oh-Jisong/secure-coding-tiny-market](https://github.com/Oh-Jisong/secure-coding-tiny-market)

목차

1. 요구사항 분석	3
2. 시스템 설계	4
3. 구현 내용	6
3.1 회원 및 상품 기능	6
3.2 채팅, 신고, 관리자 기능	7
3.3 사용자 간 포인트 송금	8
4. 시큐어 코딩 적용과 보안 약점 개선	9
4.1 인증과 권한	10
4.2 세션, 리다이렉트, 비밀키	10
4.3 HTTP 보안 헤더와 오류 처리	10
5. 체크리스트 작성 및 테스트	12
6. 유지보수 과정	13
7. 결론	14
부록. 환경 설정과 실행 방법	15

1. 요구사항 분석

과제에서 제시한 기능은 일곱 가지였다. 기능 이름만 맞추는 것보다 각 기능이 어떤 데이터와 권한을 필요로 하는지 먼저 정리했다. 예를 들어 상품 수정은 단순한 폼 제출이 아니라 작성자 또는 관리자만 가능해야 했다. 송금은 화면상 잔액 표시보다 실제 DB 변경이 함께 성공하거나 함께 실패해야 했다.

ID	기능	정상 흐름	주요 위험
R-01	회원 가입	고유 아이디, 표시 이름, 비밀번호를 입력해 계정을 만든다.	중복 아이디, 약한 입력값, 평문 비밀번호
R-02	상품 관리	상품 등록, 조회, 수정, 삭제, 판매 상태 변경	타인 상품 변조, 음수 가격, 차단 상품 노출
R-03	사용자 소통	활성 사용자 간 1:1 메시지 송수신	자기 자신 전송, XSS, 정지 사용자 접근
R-04	신고와 차단	사용자 또는 상품 신고, 관리자 승인·반려	중복 신고, 자기 신고, 권한 없는 처리
R-05	포인트 송금	활성 사용자 사이의 가상 포인트 이전	음수 송금, 잔액 초과, 일부 반영
R-06	상품 검색	제목과 판매 상태 기준 검색	SQL Injection, 와일드카드 오작동
R-07	관리자 관리	전체 사용자, 상품, 신고, 메시지, 송금 확인	일반 사용자 접근, 관리자 자기 정지

권한 원칙

일반 사용자는 자신의 데이터만 변경할 수 있다. 관리자는 운영에 필요한 범위에서 전체 요소를 조회하고 사용자 정지, 상품 차단, 신고 처리를 수행한다. 화면에서 버튼을 숨기는 것과 별개로 서버 라우트에서 권한을 다시 검사한다.

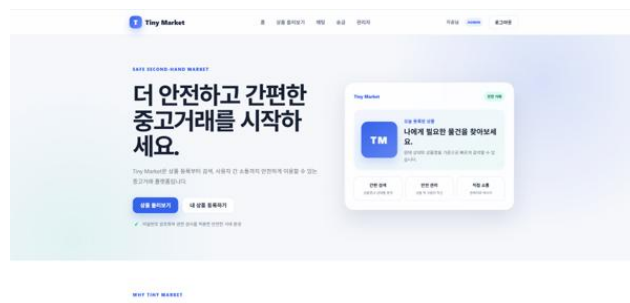


그림 1. Tiny Market 메인 화면

2. 시스템 설계

Tiny Market 시스템 구성



그림 2. 시스템 구성도

브라우저는 Jinja2 템플릿으로 렌더링된 화면을 사용한다. Flask 애플리케이션은 요청을 받아 인증 여부와 권한을 확인하고 WTForms 로 입력값을 검증한다. 데이터 접근은 SQLAlchemy ORM 을 사용했다. 비밀번호 해시와 로그인 세션은 Werkzeug 와 Flask-Login 이 담당한다. 상태를 변경하는 폼에는 Flask-WTF 의 CSRF 토큰을 적용했다.

데이터 모델 관계

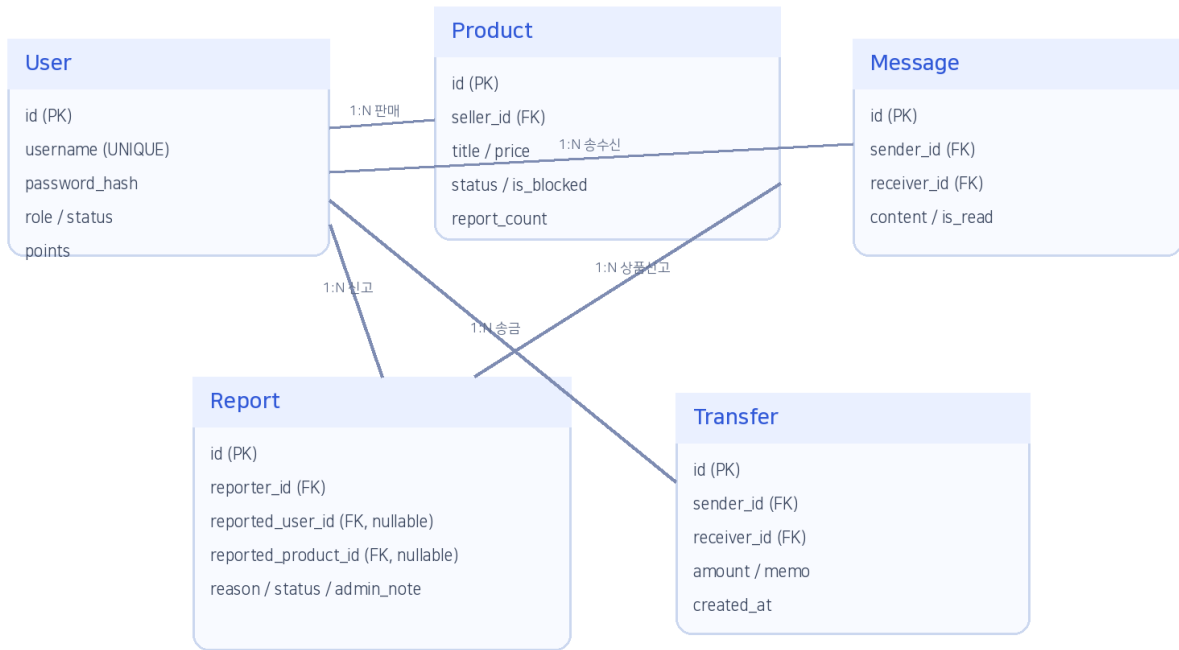


그림 3. 핵심 데이터 모델 관계

User 를 중심으로 Product, Message, Report, Transfer 가 연결된다. Report 는 사용자 또는 상품 중 정확히 하나만 대상으로 갖도록 DB 제약조건을 두었다. Message 와 Transfer 는 송신자와 수신자가 같을 수 없게 했다. User.points 와 Product.price 에도 음수 방지 제약을 추가했다.

테이블	핵심 필드	설계 의도
users	username, password_hash, role, status, points	계정 권한과 이용 상태를 한 곳에서 관리
products	seller_id, status, is_blocked, report_count	판매 상태와 운영 차단 상태를 구분
messages	sender_id, receiver_id, content, is_read	1:1 대화와 읽음 처리
reports	reporter_id, target, reason, status, admin_note	접수부터 처리까지 기록
transfers	sender_id, receiver_id, amount, memo	송금 내역과 잔액 변경 근거 보존

3. 구현 내용

구현은 회원가입과 로그인부터 시작했다. 이후 상품 CRUD 와 검색을 붙이고 채팅, 신고, 관리자 기능을 순서대로 추가했다. 마지막에 송금 기능과 오류 처리, 보안 헤더를 적용했다. 기능을 추가할 때마다 기존 화면이 깨지지 않는지 직접 확인했다.

3.1 회원 및 상품 기능

회원가입 시 아이디를 소문자로 정규화하고 중복 여부를 조회한다. 비밀번호는 generate_password_hash 로 변환한 값만 DB 에 저장한다. 로그인 실패 메시지는 아이디가 없는 경우와 비밀번호가 틀린 경우를 구분하지 않았다. 마이페이지에서는 표시 이름과 소개글을 수정하고 현재 비밀번호를 확인한 뒤 새 비밀번호를 저장한다.

상품은 제목, 설명, 가격, 이미지 URL, 판매 상태를 입력한다. 수정과 삭제는 상품의 seller_id 와 현재 사용자의 id 를 비교해 허용한다. 관리자는 운영 목적상 수정과 삭제가 가능하다. 차단된 상품이나 HIDDEN 상태의 상품은 일반 사용자에게 404 로 처리해 직접 URL 접근도 막았다.

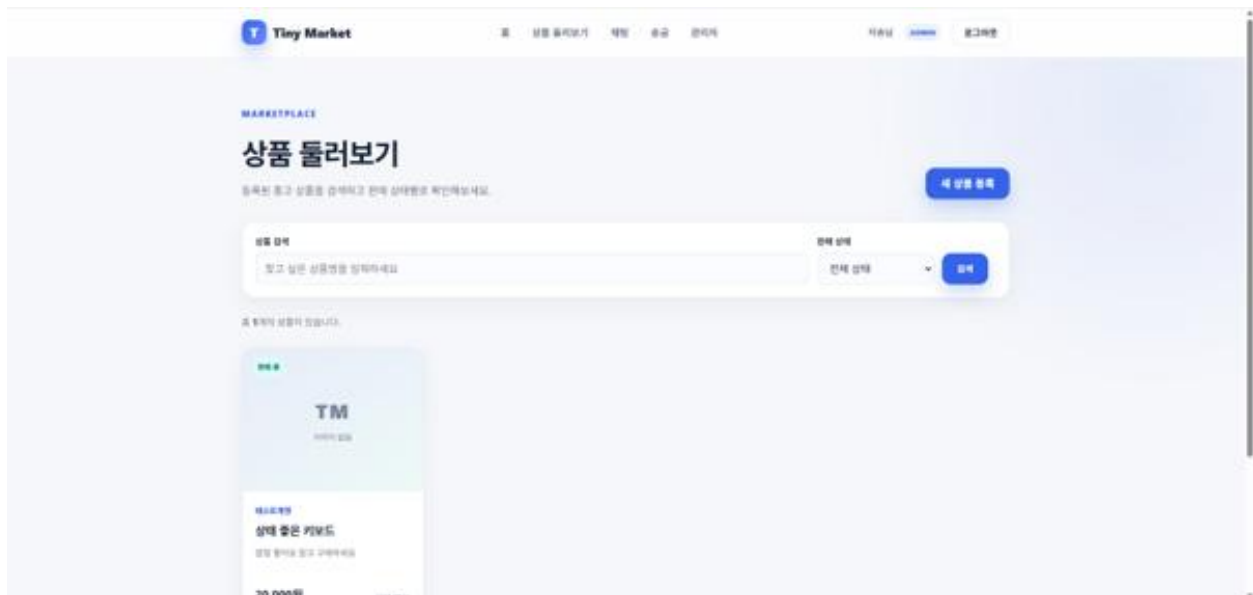


그림 4. 상품 목록과 검색 화면

3.2 채팅, 신고, 관리자 기능

채팅 목록에서는 활성 사용자만 보여준다. 두 사용자 사이의 마지막 메시지와 읽지 않은 수를 계산해 표시한다. 채팅방에 들어오면 현재 사용자가 받은 미확인 메시지를 읽음 상태로 바꾼다. 사용자 입력은 템플릿에서 HTML 로 해석하지 않아 script 태그를 보냈을 때에도 문자열로 표시된다.

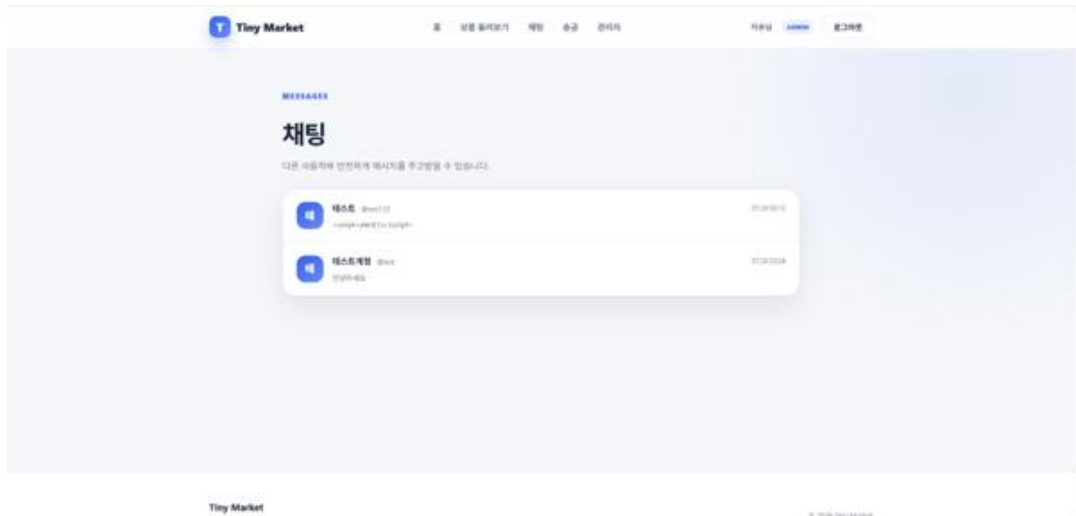


그림 5. 채팅 사용자 목록

신고는 사용자 신고와 상품 신고로 나뉘었다. 자신의 계정이나 자신의 상품은 신고할 수 없다. 같은 대상에 처리 대기 신고가 있으면 중복 접수를 막는다. 관리자는 신고를 승인하거나 반려할 수 있다. 승인 대상이 상품이면 `is_blocked` 를 `True` 로 바꾸고 사용자면 `status` 를 `SUSPENDED` 로 바꾼다.

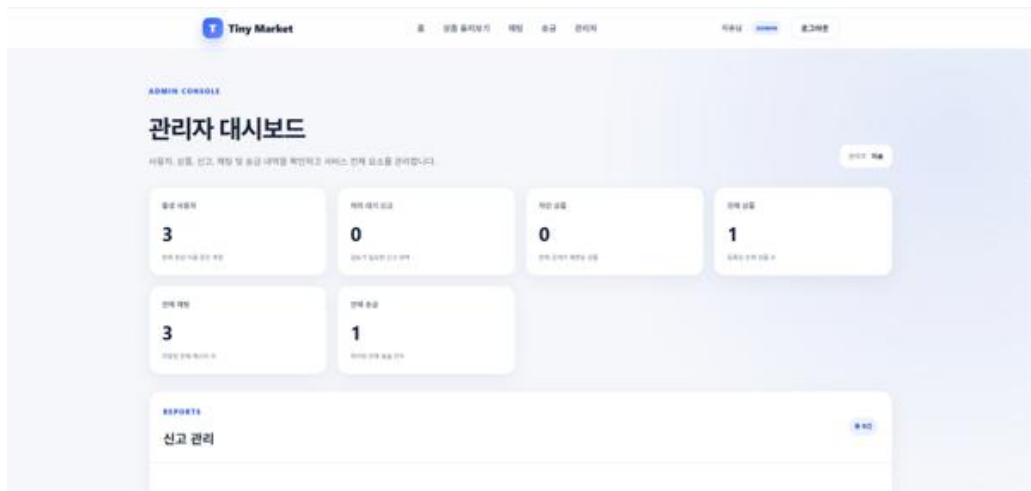


그림 6. 관리자 대시보드

3.3 사용자 간 포인트 송금

실제 결제가 아닌 플랫폼 내부 가상 포인트 기능으로 구현했다. 받는 사용자가 존재하고 활성 상태인지 확인한 뒤 송금을 처리한다. 자기 자신에게 보내는 요청과 0 이하 금액은 폼과 DB 양쪽에서 차단한다.

잔액 검사는 값을 읽은 다음 단순히 빼는 방식으로 만들지 않았다. 송신자의 points 가 송금액 이상일 때만 UPDATE 가 성공하도록 조건을 넣었다. 송신자 차감, 수신자 증가, Transfer 레코드 생성을 같은 트랜잭션에서 처리한다. 중간에 하나라도 실패하면 rollback 한다. 성공 후에는 Redirect 를 사용해 브라우저 새로고침으로 같은 송금이 다시 실행되지 않게 했다.

```
sender_result = db.session.execute(
    update(User)
    .where(User.id == current_user.id, User.points >= amount)
    .values(points=User.points - amount)
)

if sender_result.rowcount != 1:
    db.session.rollback()
else:
    # 수신자 증가와 송금 내역 저장 후 한 번에 commit
    db.session.commit()
```

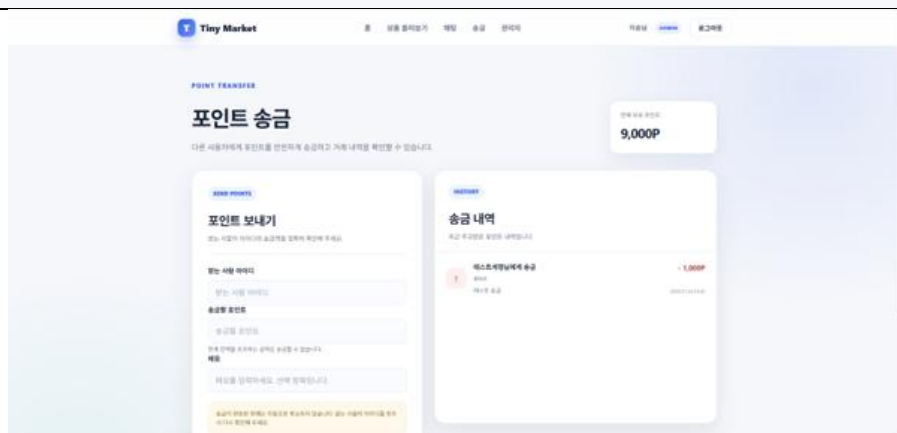


그림 7. 포인트 송금과 송금 내역

4. 시큐어 코딩 적용과 보안 약점 개선

보안 항목은 마지막에 한꺼번에 붙이지 않았다. 기능을 사용해 보면서 우회 가능한 경로를 찾고 라우트, 모델, 템플릿을 다시 수정했다. 특히 화면에 버튼이 없다는 이유만으로 권한이 보호되는 것은 아니었다. 주소와 요청 방식을 직접 바꿨을 때도 안전해야 했다.

발견한 약점	위험	수정 내용
평문 비밀번호 가능성	DB 유출 시 비밀번호 직접 노출	Werkzeug 단방향 해시 저장 및 비교
타인 상품 수정 URL 접근	상품 정보 변조	seller_id 와 current_user.id 를 서버에서 비교
상태 변경을 GET 으로 처리할 가능성	링크 클릭만으로 삭제·정지 가능	삭제, 로그아웃, 차단, 신고 처리를 POST 로 제한
CSRF 토큰 누락	로그인 사용자의 권한을 도용한 위조 요청	CSRFProtect 와 모든 POST 품의 토큰 적용
채팅·신고·메모의 HTML 입력	저장형 XSS	Jinja2 자동 이스케이프, safe 필터 미사용
문자열 기반 SQL 작성	SQL Injection	SQLAlchemy ORM 과 select 사용
검색어의 %, _ 처리	예상보다 넓은 검색 결과	contains(..., autoescape=True) 적용
잔액 확인 후 별도 차감	동시 요청 시 초과 송금	잔액 조건을 포함한 원자적 UPDATE
송금 일부만 반영	송신자와 수신자 잔액 불일치	하나의 트랜잭션과 rollback
고정 SECRET_KEY	세션 위조와 키 노출	환경변수 또는 instance 난수 파일 사용
외부 next URL 허용	Open Redirect 피싱	동일 호스트와 http/https 만 허용
정지 전 세션 유지	정지 후에도 계속 사용	before_request 에서 상태 재검사 후 로그아웃
기본 디버그 모드	내부 정보 노출	기본 실행에서 debug 비활성화
보안 헤더 부재	클릭재킹, MIME 스니핑	CSP, X-Frame-Options 등 적용

4.1 인증과 권한

관리자 라우트에는 `admin_required` 데코레이터를 적용했다. 로그인 여부를 먼저 확인하고 `role` 이 `ADMIN` 이 아니면 403 을 반환한다. 상품 수정과 삭제는 별도의 소유권 검사를 수행한다. 관리자 계정은 자신을 정지할 수 없고 다른 관리자 계정도 일반 사용자와 같은 방식으로 정지할 수 없게 했다.

```
def admin_required(view_function):
    @wraps(view_function)
    @login_required
    def wrapped_view(*args, **kwargs):
        if not current_user.is_admin:
            abort(403)
        return view_function(*args, **kwargs)
    return wrapped_view
```

4.2 세션, 리다이렉트, 비밀키

`SECRET_KEY` 는 코드에 고정하지 않았다. 운영 환경에서는 환경변수를 사용하고 값이 없으면 `instance/.secret_key` 를 난수로 생성한다. 파일 권한은 가능한 환경에서 600 으로 설정한다. 해당 폴더는 `.gitignore` 에 포함해 GitHub 에 올라가지 않게 했다.

로그인 후 이동하는 `next` 값은 현재 호스트와 같은지 확인한다. 외부 도메인으로 리다이렉트되는 경우에는 무시하고 메인 화면으로 이동한다. 세션과 Remember Me 쿠키에는 `HttpOnly` 와 `SameSite=Lax` 를 적용했다. Secure 는 production 환경에서 활성화한다.

4.3 HTTP 보안 헤더와 오류 처리

모든 응답에 `CSP`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy` 를 넣었다. 정적 파일을 제외한 응답에는 `no-store` 를 설정했다. 403, 404, 413, 500 오류 페이지를 따로 만들었다. 500 오류에서는 DB 세션을 rollback 해 다음 요청에 실패 상태가 남지 않게 했다.

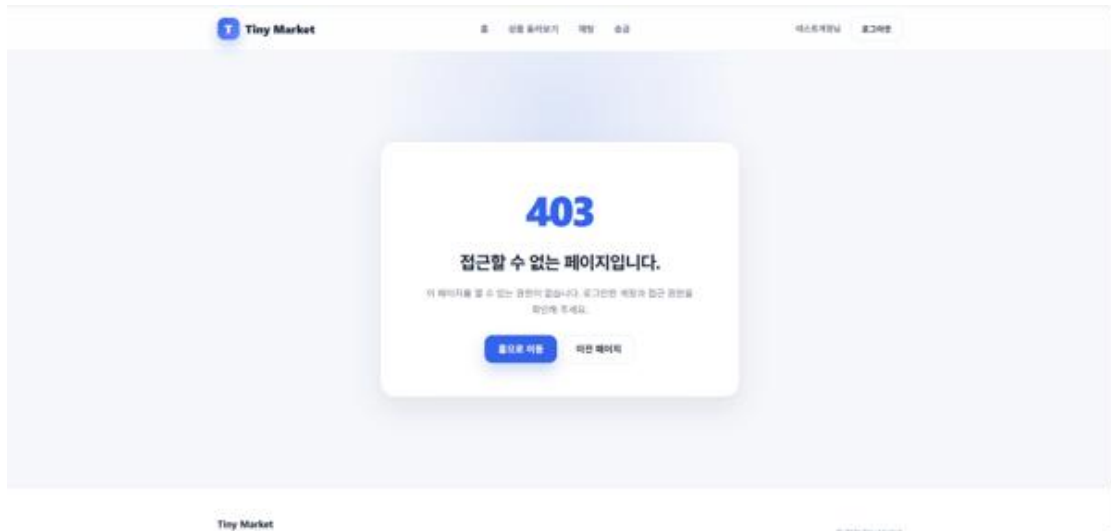


그림 8. 권한 없는 관리자 접근 시 403 화면

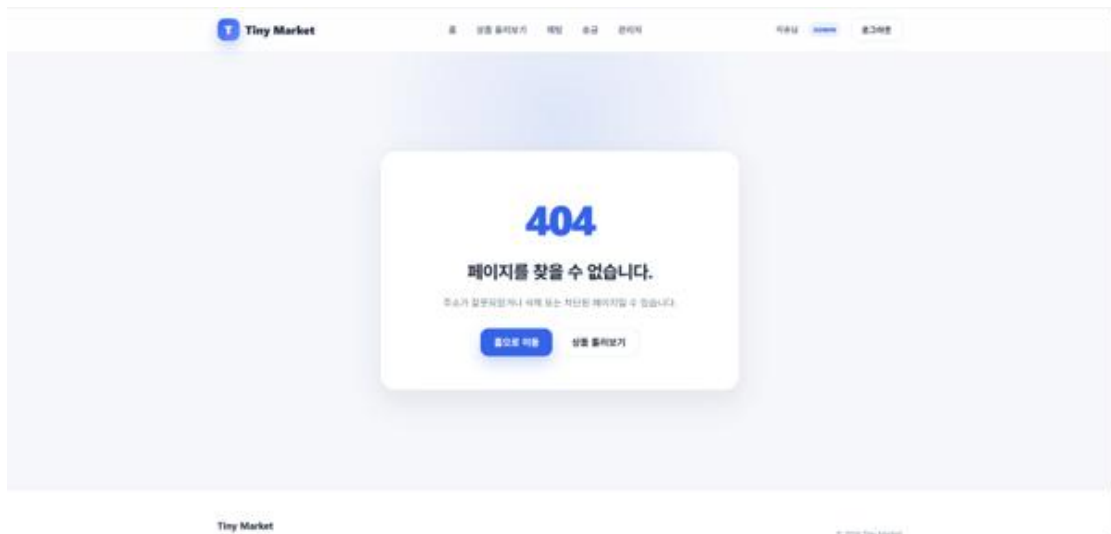


그림 9. 존재하지 않거나 차단된 페이지의 404 화면

남아 있는 제한

현재 CSP 의 script-src 에는 알림 닫기 버튼의 인라인 onclick 때문에 unsafe-inline 이 포함되어 있다. 기능은 동작하지만 더 엄격한 CSP 를 위해서는 인라인 이벤트를 별도 JavaScript 파일로 옮겨야 한다. 로그인 Rate Limiting 과 관리자 감사 로그도 운영 환경에서는 추가가 필요하다. 이 부분은 구현 완료로 과장하지 않고 향후 개선 사항으로 남겼다.

5. 체크리스트 작성 및 테스트

테스트 항목은 회원, 마이페이지, 상품, 채팅, 신고, 송금, 관리자, HTTP 보안, DB 제약조건으로 나뉘었다.

프로젝트 루트의 TEST_CHECKLIST.md 에 세부 절차와 기대 결과를 기록했다. 총 68 개 항목을 확인했고 체크리스트상 모든 항목을 통과 처리했다.

분류	대표 테스트	결과
인증	중복 아이디, 틀린 비밀번호, 정지 계정, Open Redirect	통과
상품	음수 가격, 특수문자 검색, 타인 수정·삭제, 차단 상품 직접 접근	통과
채팅	빈 메시지, 자기 자신 전송, script 입력, 읽음 처리	통과
신고	자기 신고, 중복 신고, 처리 완료 신고 재처리	통과
송금	0 원·음수·잔액 초과·자기 송금, 트랜잭션 원자성	통과
관리자	일반 사용자 403, 사용자 정지·복구, 상품 차단, 신고 승인·반려	통과
HTTP/DB	403-404-413, 보안 헤더, 쿠키 속성, DB CheckConstraint	통과

- 일반 사용자로 /admin 에 직접 접속해 사용자 정의 403 화면이 나오는지 확인했다.
- 다른 사용자의 상품 수정 URL 을 직접 입력해 서버에서 403 으로 거부되는지 확인했다.
- 채팅과 신고 상세에 <script>alert(1)</script>를 저장해 실행되지 않고 문자로 보이는지 확인했다.
- 보유 포인트보다 큰 금액, 0 원, 음수, 자기 자신, 정지 사용자 대상으로 송금을 시도해 모두 거부되는지 확인했다.
- 정상 송금 후 송신자 잔액, 수신자 잔액, 송금 내역이 함께 반영되는지 확인했다.
- 관리자가 상품 신고를 승인했을 때 상품 목록에서 사라지고 직접 URL 접근도 차단되는지 확인했다.
- curl -I 요청으로 CSP, nosniff, SAMEORIGIN, no-store 등 응답 헤더를 확인했다.

6. 유지보수 과정

확인한 문제	수정 내용
상품 목록 화면이 비어 보임	템플릿 파일 저장 과정에서 내용이 빠진 것을 확인하고 전체 파일을 다시 작성했다.
메인 화면이 상품 목록으로 바뀜	index.html 이 잘못 덮어써진 것을 확인하고 메인 히어로 화면을 복구했다.
채팅방에 사용자 신고가 없음	room.html 헤더에 상대 사용자 신고 링크와 스타일을 추가했다.
관리자 관리 범위 부족	사용자·상품·신고뿐 아니라 전체 채팅과 송금 내역 조회를 추가했다.
정지 계정의 기존 세션 유지	before_request 에서 사용자 상태를 매 요청마다 다시 확인하도록 수정했다.
고정 개발용 SECRET_KEY	환경변수와 instance 난수 파일 방식으로 교체했다.
기본 오류 화면	403, 404, 413, 500 사용자 화면을 만들고 500 에서 rollback 하도록 했다.
중복 송금 우려	성공 후 Redirect 를 적용해 새로고침 재전송을 막았다.
민감 파일 Git 업로드 위험	.gitignore 에 instance, DB, 비밀번호, 가상환경, .env 를 추가했다.

이번 과제에서 유지보수는 기능을 모두 만든 뒤 별도로 한 번 수행한 단계라기보다 개발 중 계속 반복한 과정이었다. 화면에서 발견한 누락은 템플릿으로 돌아가 수정했고 권한 우회 가능성은 라우트와 모델 제약조건을 다시 확인했다. 이 과정을 거치면서 정상 흐름만 확인하는 것보다 잘못된 사용을 의도적으로 시도하는 일이 중요하다는 것을 알게 되었다.

7. 결론

Tiny Market 은 과제에서 요구한 사용자 가입, 상품 등록과 조회, 사용자 간 소통, 악성 사용자·상품 차단, 사용자 간 송금, 상품 검색, 관리자 통합 관리 기능을 모두 포함한다. 단순히 버튼을 제공하는 데 그치지 않고 중요한 요청마다 인증, 권한, 입력값, 대상 상태를 서버에서 다시 확인했다.

가장 신경 쓴 부분은 송금과 관리자 권한이었다. 송금은 잔액 차감과 증가가 분리되지 않도록 트랜잭션으로 처리했고 관리자는 별도 데코레이터와 서버 검사를 통해 일반 사용자가 접근할 수 없게 했다. 완성 후에는 정상 기능과 공격성 입력을 함께 확인하는 체크리스트를 작성했다.

운영 서비스 수준으로 확장하려면 HTTPS 배포, 로그인 Rate Limiting, 관리자 감사 로그, 데이터베이스 마이그레이션, 자동 테스트와 CI 가 추가로 필요하다. 현재 프로젝트는 교육용 로컬 애플리케이션 범위에서 보안 약점을 줄이는 데 초점을 맞췄다.

GitHub 저장소

<https://github.com/Oh-Jisong/secure-coding-tiny-market>

부록. 환경 설정과 실행 방법

README.md 와 동일한 핵심 절차를 요약했다.

1) 저장소 복제

```
git clone https://github.com/Oh-Jisong/secure-coding-tiny-market.git  
cd secure-coding-tiny-market
```

2) 가상환경 생성 및 활성화

```
python3 -m venv venv  
source venv/bin/activate
```

3) 의존성 설치

```
pip install --upgrade pip  
pip install -r requirements.txt
```

4) 애플리케이션 실행

```
python app.py
```

5) 브라우저 접속

```
http://127.0.0.1:5001
```

관리자 계정은 일반 회원가입 후 DB 에서 해당 계정의 role 을 ADMIN 으로 변경해 사용한다. 기본 관리자 비밀번호를 소스에 넣지 않았다. 최초 실행 시 users, products, messages, reports, transfers 테이블과 instance/tiny_market.db 가 생성된다.